

# C++ Handbook

- #include <bits/stdc++.h> → already includes most of the essential header files (iostream etc.)
- Buffer: when you use `cout`, program does NOT immediately print text on screen. It puts it in a temporary memory BUCKET (ie a buffer). When the bucket is filled, text is shown on the screen altogether.  
(\*) This takes way less time & effort than individually performing the task.
- FLUSH: The act of forcing the bucket to empty out onto the screen RIGHT NOW even if it isn't completely full is called FLUSHING.
- Input & Output are sometimes the bottleneck of the program (ie they make it slow due to large i/o values). Writing the following lines at the beginning of the program makes it more efficient.

```
ios :: sync_with_stdio (0);  
cin.tie (0);
```



• ios::sync\_with\_stdio(0); → C++ is built on top of older C.

I/O methods are

C++ : cout & cin

C : printf & scanf

Now in general, C++ SYNCHRONIZES these two buckets to make sure that if you mix in b/o, i.e. in one line you say cout then printf then again cout, C++ makes sure it knows what bucket to empty when.

However this is extra work & slows down the program. So we pass '0' to sync\_with\_stdio i.e. false to break this synchronization.

(\*) You promise that you won't use printf & scanf and you will stick to cin & cout.

• cin.tie(0); → cin & cout are "tied" together for user comfort.

i.e. say  
cout << "Enter age: ";  
cin >> age;

"Tied" means that before cin is called, automatically flush operation empties the bucket so that user sees the question and knows what he/she has to do.



In cp however, you're probably never going to print question on screen & besides the computer is checking for testcases so it doesn't even matter if the question was shown.

In other words, if `cin.tie(0);` is used before that age program, question "Enter age" would probably not even be displayed on the screen & it would just be there waiting for input.

Clearly in cp all of this is not required so passing false i.e. "0" to `cin.tie` "unties" `cin` & `cout`.

Now there is no needless emptying of output buffer & this saves execution time.

- Use "\n" in place of "endl": Again, `endl` causes a flush operation so `\n` is much more efficient.

- `string s;`  
`getline (cin, s);` } If you want to take a sentence input with spaces.

- `while (cin >> x) {`  
    //code  
    }  
} } Useful if amount of data is unknown.  
(It will terminate when it encounters "End of File")  
(Very ez in cp bcoz list of all testcases is a file)  
manually, `Ctrl + Z` then `Enter` will terminate

Also reminds if diff datatype than input



- If a contest uses files for i/o, just add these 2 lines in beginning.  
write all your code as usual.

```
freopen ("input.txt", "r", stdin);  
freopen ("output.txt", "w", stdout);
```

r : read      w : write

(\*) So currently this is how you'll write a general C++ program for a contest on say codeforces.

```
#include <bits/stdc++.h>  
using namespace std;
```

```
int main () {
```

```
ios :: sync_with_stdio (0);
```

```
cin.tie (0);
```

```
long longll t; cin >> t;
```

```
while (t--) {  
    // code here
```

```
}
```

```
}
```

→ add this here

type def

long long

ll;



- int : 32 bit  $-2^{31}$  to  $2^{31}-1$   
 $(\sim 10^9)$  or  $-2 \times 10^9$  to  $2 \times 10^9$
- long long : 64 bit  $-2^{63}$  to  $2^{63}-1$   
 $(\sim 10^{18})$  or  $-9 \times 10^{18}$  to  $9 \times 10^{18}$

(\*) long long x = 123456789123456789LL

add an LL while defining as right side is checked first so it assumes int.

- You won't need anything bigger than long long and mostly contexts don't even support it but a 128 bit also exists -

-- int 128 -t  $-2^{127}$  to  $2^{127}-1$   
 $(\sim 10^{38})$  or  $-10^{38}$  to  $10^{38}$

- Sometimes the final answer may be too large to report & check so questions uses MODULAR ARITHMETIC.

They'll ask you answer %  $(10^9+7)$ .

So answer becomes small enough to fit in int

Properties:

$$(a+b) \% m = (a \% m + b \% m) \% m$$

$$(a-b) \% m = (a \% m - b \% m) \% m$$

$$(a \times b) \% m = (a \% m \times b \% m) \% m$$

for subtraction involved cases, you should check for negative and add m.



(\*) Suppose you need to compute  $100!$   
You won't be able to fit it in  
long long. So for remainder, you'll  
still have to get remainder at  
each step as you can't actually  
store  $100!$  anywhere.

```
long long x = 1LL;  
for (int i = 2; i <= 100; i++) {  
    x = (x * i) % m;  
}  
cout << x % m << "\n";
```

So at each step take mod.

(\*) As a precaution, always check  
for negative remainder and  
add an  $m$  to it.

• double  $\rightarrow$  64 bit      long double  $\rightarrow$  80 bit

• when comparing floating pt. nos., there  
might be precision errors.

```
if (abs(a-b) < 1e-9) {  
    }  
}
```

so this  
is better  
way to compare

$$\epsilon = 10^{-9}$$



- Now, it kinda makes sense to use long long always you know just in case. So you can shorten long long as it is, well, too "long" to write.

```
typedef long long ll;
```

```
ll a = 123456789LL;
```

```
ll b = 987654321LL;
```

(X) This was "type name".

Another thing is MACROS

```
#define PB push-back
```

now you won't have to write the string "push-back" anywhere in the code.

Just PB would work.



# TIME COMPLEXITY:

$O(\dots)$

a function of 'n'  
the size of input.

if input is array,  $n \rightarrow$  no. of elements

if string  $n \rightarrow$  length of string

- Nested loops  $\rightarrow O(n^k)$

for (int i=0; i<n; i++)  $\Rightarrow O(n)$

Think of it as basically going through  
the code 'n' times.

- $O(n^2)$  for 2 nested loops ( $i < n, j < n$ )

- $O(mn)$  if ( $i < n, j < m$ )

- We only consider order of magnitude.

$i < 3n \rightarrow O(n)$  [actually  $3n$ ]

$i < n; i += 2 \rightarrow O(n)$  [actually  $\lceil n/2 \rceil + 1$ ]

- if there are "phases" i.e. first code block  
 $O(n)$  then  $O(n^2)$  then  $O(n)$   
net is just  $O(n^2)$   
(Think of it like sum of these & n very large)



- for recursion, product of no. of function calls & time complexity for one call.

f  $\Rightarrow$ 

```
void f (int n) {
    if (n==1) return;
    f(n-1);
}
```

 $\rightarrow$   $O(n)$   
 $n \times O(1)$

g  $\Rightarrow$ 

```
void g (int n) {
    if (n==1) return;
    g(n-1);
    g(n-1);
}
```

 $\rightarrow$   $g(n) \rightarrow 1$   
 $g(n-1) \rightarrow 2$   
 $g(n-2) \rightarrow 4$   
 $\vdots$   
 $g(1) \rightarrow 2^{n-1}$   
 $1 + 2 + 2^2 + \dots + 2^{n-1} = 2^n - 1$   
 $\rightarrow$   $O(2^n)$

- $O(1) \rightarrow$  Const. time algo; indep of  $n$ .  
eg: Direct formula like  $\frac{n*(n+1)}{2}$

- $O(\log n) \rightarrow$  logarithmic algo; half input at each step.  
 $\downarrow$   
fastest after 1.  
 $\log_2 n =$  no. of times  $n$  must be divided by 2 to get 1

- $O(\sqrt{n}) \rightarrow$  Slower than " $\log n$ " but faster than " $n$ "  
 $\log n < \sqrt{n} < n$   
for large  $n$ .  
 $\sqrt{n} = \frac{n}{\sqrt{n}}$  so  $\sqrt{n}$  lies somewhere in middle of  $n$ .  
Eg: Check if no. is prime, check factors as far as  $\sqrt{n}$  repeat



- $O(n)$  → linear ; best possible in most cases as usually 1 pass for some input determine note hai.

- $O(n \log n)$  → **Sorting complexity** (efficient sorting)  
or an algo using data structure where each operation takes  $O(\log n)$  time.

- $O(n^2)$ ,  $O(n^3)$  just 2 & 3 nested loops.

- $O(2^n)$  → Iterating through **all subsets** of inputs

$$\{1, 2, 3\} \Rightarrow \phi, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\} \rightarrow 2^3 = 8.$$

- $O(n!)$  → time complexity for iterating through **all permutations**

eg.  $\{1, 2, 3\}$

$\begin{matrix} 123 \\ 132 \\ 213 \\ 231 \\ 312 \\ 321 \end{matrix}$ 
}  $\rightarrow 6 = 3!$

**Chronology**

Best →  $O(1)$

$O(\log n)$

$O(\sqrt{n})$

$O(n)$

$O(n \log n)$

$O(n^2), O(n^3)$

Worst →

$O(n!), O(2^n)$



• Polynomial Time Complexity:  $O(\dots) \leq O(n^k)$  for some  $k$ .

only  $O(2^n)$  &  $O(n!)$  are not polynomial for the discussed cases.

(\*) NP-Hard Problems are ones for which no polynomial T.C. is known.

# Estimating Efficiency:

$10^8$  operations / sec  
↑  
Golden Rule

Time limit in general is 1 second.

Say  $n = 5$  &  $O(n^2)$  so  $10^{10}$  ops

$\therefore O(n^2)$  will TLE.

For 1 sec

| <u>input</u>                  | <u>T.C.</u>             |
|-------------------------------|-------------------------|
| $n \leq 10$                   | $O(n!)$                 |
| $n \leq 20$                   | $O(2^n)$                |
| $n \leq 500$                  | $O(n^3)$                |
| $n \leq 5000$                 | $O(n^2)$                |
| (*) $\rightarrow n \leq 10^6$ | $O(n \log n)$ or $O(n)$ |
| $n$ very large                | $O(1)$ or $O(\log n)$   |

so next time you see  $n \leq 10^5$  you know  $O(n)$  or  $O(n \log n)$  is expected.



Max Subarray Sum: 3 possible algo.  
 $O(n^3)$ ,  $O(n^2)$ ,  $O(n)$

Most efficient  $O(n)$  Kadane's Algo:

$a_1, a_2, a_3, a_4, \dots, a_n$

↑  
either  $[a_3]$   
or  $[M(a_1, a_2), a_3]$

in gen. at any  $a_i$

max sum is  $\max \text{sum}(a_1, a_2, \dots, a_{i-1}) \& a_i$

or only  $a_i$

Basically either include  $a_i$  or start a new one.

```
int best = 0, maxsum = 0;  
for (int i = 0; i < n; i++) {  
    maxsum = max(a[i], maxsum + a[i]);  
    best = max(best, maxsum);  
}  
cout << best << "\n";
```

$O(n)$  sol<sup>n</sup>

$n \leq 10^6 \rightarrow 1 \text{ sec.}$



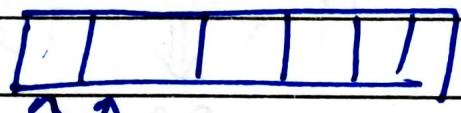
# Sorting: Many times problem can be solved easily if you sort first.

Efficient Sorting is  $O(n \log n)$ .

• Simple algs are  $O(n^2)$

(\*) Bubble sort: move largest to last then 2<sup>nd</sup> largest & to an

$n-1$  steps.  
then  $n-2$   
:  
:  
1  
total  
 $1+2+\dots+(n-1)$



1 3 8 2 9 2 5 6  
 $\uparrow \uparrow \uparrow \uparrow$   
ok ok  $\downarrow$  swap

So in  $O(n)$   
you find largest

$n$  times  $O(n)$   
for all elements

$\frac{n(n-1)}{2} = O(n^2)$  1 3 2 8 9 2 5 6  
 $\uparrow \uparrow$   
ok  $\downarrow$  swap

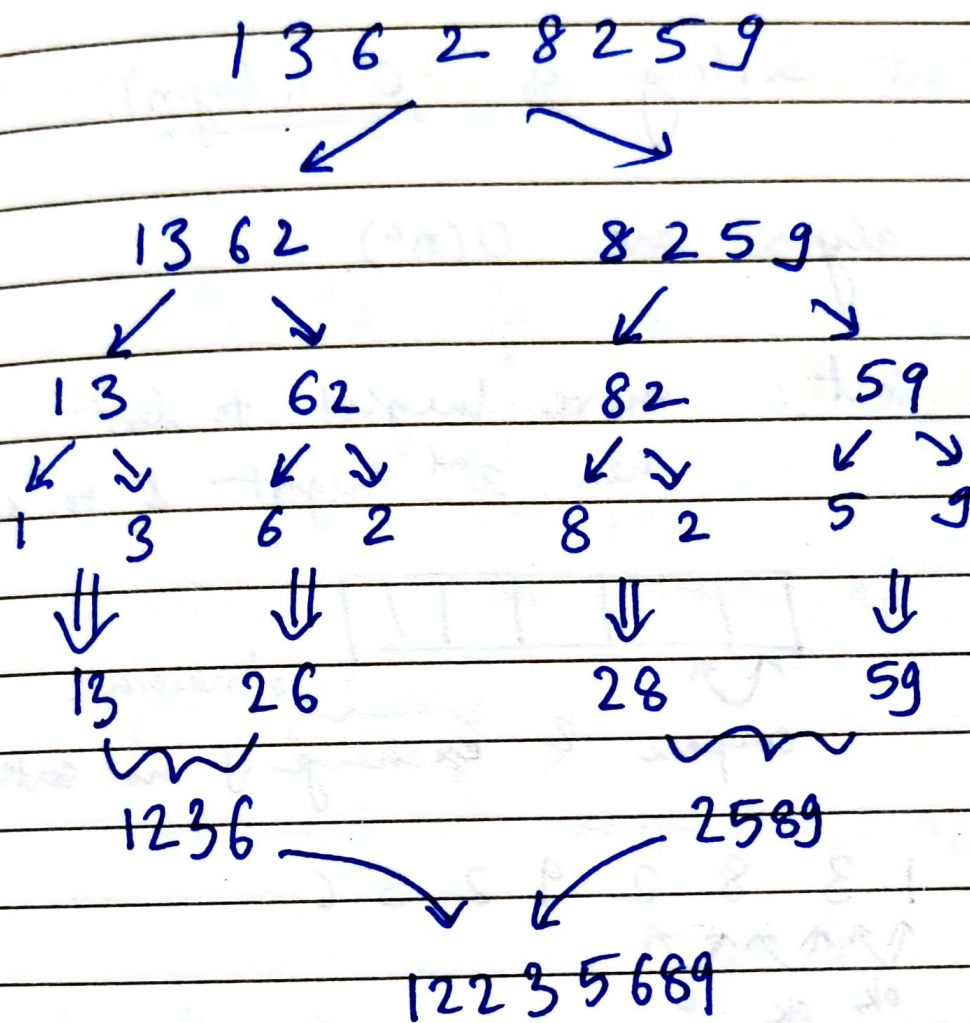
Bubble sort

$\sim O(n^2)$

```
for (int i=0; i<n; i++) {  
    for (int j=0; j<n-1; j++) {  
        if (a[j] > a[j+1]) {  
            //swap  
        }  
    }  
}
```



$O(n \log n) \rightarrow$  Merge Sort:



Recursion has  $\log n$  levels.

Each merging can be done in  $O(n)$

So  $O(n \log n)$



```

Void mergeSort (int seq[], int l) {
    if (l == 1) return;
    int p1 [l/2], p2 [l - l/2];
    for (int i=0; i < l/2; i++) p1[i] = seq[i];
    for (int i=0; i < l-l/2; i++) p2[i] = seq[i+l/2];

    mergeSort (p1, l/2);
    mergeSort (p2, l-l/2);
    merge (p1, l/2, p2, l-l/2, seq);
}

```

```

void merge (int p1[], int l1, int p2[],
            int l2, int seq[]) {
    int p1-front = 0, p2-front = 0;

```

```

    for (int i=0; i < l1+l2; i++) {
        if (p1-front < l1 && p2-front < l2) {
            if (p1[p1-front] < p2[p2-front]) {
                seq[i] = p1[p1-front];
                p1-front++;
            }
            else {
                seq[i] = p2[p2-front];
                p2-front++;
            }

```

```

        }
        else if (p1-front < l1) { seq[i] = p1[p1-front]; p1-front++; }
        else { seq[i] = p2[p2-front]; p2-front++; }
    }
}

```



NOT Possible to get sorting efficiency better than  $O(n \log n)$

$a_1 a_2 a_3 \dots a_n$

$n$  elements of array you need to sort.

Total possible states are  $n!$

$a_i > a_j$  is 1 piece of information.

So decision tree will have  $\log_2(n!)$  height.

$$\log_2(n!) \geq \frac{n}{2} \log_2 \frac{n}{2}$$

$$\text{So } O(n \log n)$$



## Predef sorting fuctions

① for vector  $v \rightarrow \text{sort}(v.begin(), v.end());$   
for decreasing order  $\rightarrow \text{sort}(v.rbegin(), v.rend());$

② for array  $a \rightarrow \text{sort}(a, a+n);$

③ for string  $s \rightarrow \text{sort}(s.begin(), s.end());$

(\*) pairs are sorted acc. to first then second.  
 $(1,2) < (1,5) < (2,3)$

(\*) tuples also similarly.  
 $(1,5,3) < (2,1,3) < (2,1,4)$

- Suppose you define a structure Point  
ic  $(x,y)$ . then to sort it the comparison  
operator should have some meaning.  
Hence, you'll have to custom define  
how you want to sort.  
Say first priority to  $x$  then  $y$ .

```
struct P {  
    int x, y;  
    bool operator < (const P &p) {  
        if (x != p.x) return x < p.x;  
        else return y < p.y;  
    }  
};
```



- You can also define custom comparison functions.

By default, sort function uses the default meaning of  $>$  or  $<$ .

Suppose

```
bool comp (string a, string b) {  
    if (a.size() != b.size()) {  
        return a.size() < b.size();  
    }  
    return a < b;  
}
```

This will compare length & if it is same, then alphabetical order.

So. `sort (v.begin(), v.end(), comp)`

like this you can use your own comparison.



## # BINARY SEARCH:

In general if you want to find  $a_i$  in  $a_1, a_2, \dots, a_n$ , simplest method is  $O(n)$

Just check if current element ==  $a_i$ .  
This is fastest for a random array

Suppose array is sorted.

Then Bin Search sol<sup>n</sup> is  $O(\log n)$  ie faster.

```
int first = 0, last = n - 1;
while (first <= last) {
    int mid = (first + last) / 2;
    if (a[mid] == x) {
        // x found at index mid
    }
    if (a[mid] > x) last = mid - 1;
    else start = mid + 1;
}
```

Act approach: make jumps of  $n/2, n/4, n/8$  until you reach  $x$ .

$O(\log n)$

```
int start = 0;
for (int jump = n/2; jump >= 1; jump /= 2) {
    while (start + jump < n && a[start + jump] < x) {
        start += jump;
    }
    if (a[start] == x) {
        // found
    }
}
```



Now while solving a problem, you wouldn't want to write your bin search logic as you may make mistakes.

- lower-bound ( $a, a+n, x$ )  $\rightarrow$  returns a pointer  $\{ \geq x \}$  to first array element with value at least  $x$ .
- upper-bound ( $a, a+n, x$ )  $\rightarrow$  first array element with value greater than  $x$   $\{ > x \}$
- equal-range ( $a, a+n, x$ )  $\rightarrow$  gives both above pointers as pair datatype (just use auto)  
(• first  $\rightarrow$  l.b. • second  $\rightarrow$  u.b.)

(\*) On subtracting these pointers from 'a' you can get index.  $((\text{pointer}) - (\text{pointer}) = \text{int})$   
subtraction

(\*) All of this is bin search only so  $O(\log n)$  & no code for you to write.



```

auto k = lower-bound(a, a+n, x) - a;
if (k < n && a[k] == x) { // found

```

```

auto a = lower-bound(a, a+n, x);
auto b = upper-bound(a, a+n, x);
b - a → no. of elements with value x.

```

Same code with equal-range:

```

auto s = equal-range(a, a+n, x);
s.second - s.first → no. of elements with
value = x.

```

## APPLICATIONS OF BIN SEARCH:

Suppose you're asked "minimum time" or "max machines" etc type of ques & you're getting TLE with your algo.

You could write a function bool valid (ans) to simulate. So for time it'll be something like this

| time (s) | 1     | 2     | 3     | 4    | 5    | 6    | 7    | ... |
|----------|-------|-------|-------|------|------|------|------|-----|
| Valid    | false | false | false | true | true | true | true | ... |

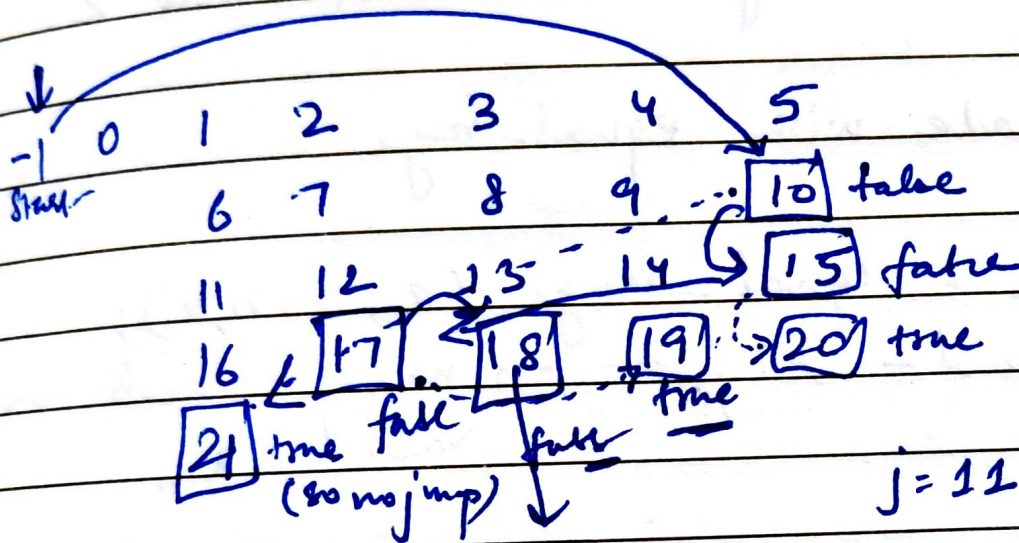
so min is 4 right.



What you can do is  
stand at -1

fix your jump size to something  
like  $10^9$  (check how large could  
your answer be)

now jump if valid ( $10^9$ ) is false  
if its true just half your jump size.



18 is Not true

19 is

```
int position = -1;
for (int jump = 2; jump >= 1; jump /= 2) {
    while (!valid(position + jump)) {
        position += jump;
    }
}
int answer = position + 1;
```

position becomes largest possible false

so position + 1 is the first true  
or minimum time we required.



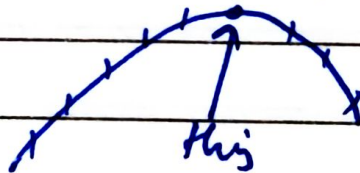
(\*) Advisable to keep 'Z' ie initial jump very large. (eg: some value you know is 'true'.)

(\*) outer loop is  $O(\log Z)$   
valid function will have some T.C. say  $O(n)$

then  $O(n \log Z)$   
So make valid function accordingly.

# Another application.

Sometimes you're asked info related to max value of a certain function.



$O(\log n)$  is again useful.

```
int x = -1;  
for (int b = Z; b >= 1; b /= 2) {  
    while (f(x+b) < f(x+b+1)) x += b;  
}  
int ans = x+1;
```

NOTE: here consecutive values can't be equal tho.